

I. BACKGROUND

A. DCS - Digital Control System

Before the wide scale availability of digital computer hardware, control systems were implemented in hardware and utilized the system's physical response to their environment as a means of basic control [?, p 4-6]. The utility of using digital computing to provide system control is apparent, digital computers are flexible, typically fast (relative to actuation) and in recent times cheap relative to a physical control system. "The use of physical computers in the loop yields the following advantages over analog systems: (1) reduced cost, (2) flexibility in response to design changes, and (3) noise immunity" [?, p. 579]. The size of computer hardware has substantially decreased in recent decades, and "[w]ith the development of the minicomputer in the mid-1960's and the microcomputer in the mid-1970's, physical systems no longer need to be controlled by expensive mainframe computers" [?, p. 578].

1) *PLC - Programmable Logic Controller*: In industrial control, DCSs are often implemented using PLCs, which "[are] digital, electronic device[s] designed to control machines and processes by performing event-driven and time-driven sequential operations. [They] can be programmed without special programming skills, and [they] can be maintained by factory maintenance technicians" [?, p. 432]. Many PLCs are configured to communicate with a distributed control system [?, p. 395]. "Programmable logic controllers (PLCs) can be considered to have three parts: the input/output section, the processor, and the programming device, or terminal" [?, p. 76]. As discussed in Section ?? PLCs are typically more expensive than consumer grade controllers (such as the Arduino AVR board family).

2) *Arduino AVR - ATmega RISC Controller*: Arduino is a widely used low-cost, consumer grade microcontroller family, notably containing the Arduino Uno Rev3 [?] and the Arduino Mega 2560 Rev3 [?]. These two controllers have many features that can be used to configure logical control loops for most simple applications:

- 1) Digital I/O Pins
- 2) Analog Input Pins (ADC) (can be used as Digital I/O)
- 3) Digital PWM Output Pins
- 4) 8-bit and 16-bit Timers
- 5) Timer Overflow and Comparison Interrupts
- 6) Digital Pin Driven Interrupts

These boards do have severe hardware limitations that prevent the use of complex programs as they feature low frequency microcontrollers (ATmega series) and very limited memory (model dependent). These boards can be programmed using the Arduino IDE or the Arduino IDE CLI tool (which can be called from another program) [?].

3) *Data Collection*: Typically Digital Control Systems both need to collect data from their environment and configure pin outputs to enact actuation of the equipment they control. "To get samples of a physical signal such as a position or velocity into digital form, we typically have a sensor that

produces a voltage proportional to the physical variable and an analog-to-digital converter" [?, p. 102]. In the case of mechanical boolean inputs such as buttons and switches, or even electrically or mechanically thresholded analog inputs, digital I/O pins can be used for boolean data collection. Data collection using a digital controller will be subject to several key limitations:

1. **Sensor Calibration Errors**: Firstly the data being collected by the DCS can itself be erroneous, electronic or mechanical sensors, especially budget friendly or consumer quality sensors, can themselves be inaccurate. Depending on sensor position relative to the controller, and the quality and thickness of the wires connecting it to the controller, interference may also disrupt the signal.
2. **Controller Calibration Errors**: The controller itself may also be miscalibrated, such that even accurate sensor readings will be incorrectly recorded. An example of this error type would be zero offset error [?, p. 334] which occurs when an analog input is arbitrarily offset during conversion.
3. **Sample and Hold**: Sample and hold involves polling an input at uniformly separated times t seconds (or ms) a part, these samples are the assumed value of that input for the remainder of the next duration t [?, p. 103]. This results in sampled data which unlike truly analog data, can be stored using finite storage space.
4. **Sampling Rate**: The sampling rate of a given sensor must be at least twice the value's expected maximum changing frequency to avoid aliasing [?, p. 103-109]. This provides a lower limit for practical sampling rate.
5. **Quantization**: Like sampling, quantization is a method of data reduction [?, p. 324] [?, p. 332]. When a signal is quantized, one chooses n bits to represent each sample which allows 2^n discrete values to be distinguished. The signal is then *rounded* to one of these values. Whether this rounding is performed via closest match, round-up or round-down depends on the hardware specifics of the Analog-to-Digital Converter being used.

From this it should be observed that any data recorded by the controller itself is subject to scrutiny meaning any data logging performed by a supervisory system will have data limitations and will likely also have data errors.

B. SCADA - Supervisory Control and Data Acquisition

Supervisory Control and Data Acquisition refers to a system that can both implement control over some process(es) and collect data from said process(es). "The digital computer can perform two functions: (1) supervisory[,] external to the feedback loop; and (2) control[,] internal to the feedback loop" [?, p. 578]. There are two major components to a SCADA system, as outlined below. For the purposes of this project, SCADA systems will be considered in the context of controlling one or more DCSs (using an Arduino AVR board) and collecting data from those DCSs. Since this system should be able to run on standard consumer hardware, it will consist of only software.

1) *Supervisory Control*: To implement software control over an Arduino AVR powered DCS, communication with the DCS must be established. This can be achieved over USB serial as Arduino AVR boards have USB serial capabilities [?]. There are two possible implementations for this control, they are named in accordance with how they function:

1. **Flash Control**: An instruction from the SCADA software is converted into Arduino IDE compatible C++ code and uploaded to the Arduino. This instruction is persistent and will survive unpowering of the DCS.
2. **Serial Control**: An instruction from the SCADA software is sent over USB serial as a string which is interpreted by the Arduino. This instruction is volatile and will be reset to the default value if the controller loses power.

Though it may be possible to enact supervisory control over an Arduino AVR board by other means than USB serial, such considerations are outside the scope of this project.

2) *Data Acquisition*: Data acquisition from the perspective of the supervisory system is subject to key limitations:

1. **Serial Bandwidth**: A serial connection to the DCS can only transmit a limited amount of data in each direction over each second. This can be modulated by baud rate and polling delay, however the amount of data that can be transmitted is limited by the USB serial controller on the DCS as well as the USB serial controller on the SCADA system's host machine.
2. **Polling Rate**: The supervisory system must poll each DCS for data at some interval T , each T all available packets are obtained and stored. The rate this can be achieved will be limited by the speed of the database being used to store data (if data storage is implemented) and the duration of time needed for all variables to be transmitted from the DCS to the supervisor.
3. **Packet Corruption**: If the supervisory system beings polling a connected DCS during transmission of a packet (containing some data, maybe a variable and its current value), only part of this packet is captured leading to data corruption.
4. **Packet Loss**: If the serial connection is overburdened (likely from excessive baud rate or insufficient DCS-side transmission delay) packets may be lost.

The result of these limitations is that the data stored by the supervisory system will likely have missing or corrupted points in each series, and may miss sudden changes in DCS sensor states (thus not recording those events).